BSc (Hons) Artificial Intelligence and Data Science

COURSEWORK REPORT

Module:
CM2605 - Simulation and Modelling Techniques

Module Co-ordinator:

Mr. Prashan Rathnayaka

RGU STUDENT ID	**2425578**
IIT STUDENT ID	**20240184**
STUDENT NAME	**Jayalath Arachchilage Nethmi Vishara Jayalath**

CW Part 01 — Simulation and Stochastic Analysis of Hospital Emergency Resources

Introduction

This report presents a discrete-event simulation of the Emergency Room (ER) triage process at a hospital experiencing overcrowding during its weekend shifts. The simulation models patient arrivals and service provided using appropriate stochastic distributions, and evaluates whether the current staffing level of 3 triage doctors meets the hospital board's mandated threshold of an average patient waiting time not exceeding 10 minutes.

A second scenario is also simulated with 4 triage doctors to assess and compare whether the additional resource resolves the overcrowding issue.

Model Parameters

The system is modelled as an **M/M/c queue**, where:

- Arrivals follow a **Poisson process** with rate $\lambda = 12$ patients/hour ($= 0.2$ patients/minute)
- Service times follow an **Exponential distribution** with mean $\mu = 12$ minutes per patient per doctor
- Queue discipline is **First-Come, First-Served (FCFS)**
- Number of servers $c = 3$ (Scenario 1) and $c = 4$ (Scenario 2)
- We simulate $n = 500$ patients

The server utilisation factor is:

$$\rho = \frac{\lambda}{c \cdot \mu^{-1}} = \frac{12 \text{ patients/hr}}{c \times 5 \text{ patients/hr/doctor}}$$

For $c = 3$: $\rho = \frac{12}{15} = 0.80$

For $c = 4$: $\rho = \frac{12}{20} = 0.60$

Model Input Parameters

```
arrival_rate <- 12            # patients per hour
service_time_mean <- 12       # minutes
service_rate <- 60 / service_time_mean  # patients per hour

num_doctors <- 3
n_patients <- 500
```

Scenario 1: Current System (3 Triage Doctors)

```
set.seed(123) # ensures reproducibility

# Step 01 - Generate Arrivals

# Generate interarrival times (hours)
interarrival_times <- rexp(n_patients, rate = arrival_rate)

# Compute arrival times
arrival_time <- cumsum(interarrival_times)

# Step 02 - Generate Service Times

# Generate service times (hours)
service_time <- rexp(n_patients, rate = service_rate)

# Step 03 - Initialize Tracking Variables

# Initialize vectors
service_start <- numeric(n_patients)
service_end <- numeric(n_patients)
waiting_time <- numeric(n_patients) # time spent in queue before service
system_time <- numeric(n_patients) # total time in system (wait + service)

# Track availability of each doctor
doctor_available_time <- rep(0, num_doctors)

# Step 04 - Simulation Loop (FCFS Logic)

for (i in 1:n_patients) {

  # Find doctor who becomes available first
  next_doctor <- which.min(doctor_available_time)

  # Service starts at max(arrival, doctor availability)
```

```r
  service_start[i] <- max(arrival_time[i], doctor_available_time[next_doctor]
)

  # Service ends after service time
  service_end[i] <- service_start[i] + service_time[i]

  # Update doctor's next available time
  doctor_available_time[next_doctor] <- service_end[i]

  # Waiting time (queue time)
  waiting_time[i] <- service_start[i] - arrival_time[i]

  # Total time in system
  system_time[i] <- service_end[i] - arrival_time[i]
}

# Step 05 - Store Results

# Combine into a dataset
simulation_data <- data.frame(
  Patient_ID = 1:n_patients,
  Arrival_Time = arrival_time,
  Service_Start_Time = service_start,
  Service_End_Time = service_end,
  Waiting_Time = waiting_time,
  System_Time = system_time
)

# Display first 10 rows
knitr::kable(
  head(simulation_data, 10),
  caption = "First 10 Patients in Simulation (M/M/c Queue)",
  digits = 2
)
```

First 10 Patients in Simulation (M/M/c Queue)

Patient_ID	Arrival_Time	Service_Start_Time	Service_End_Time	Waiting_Time	System_Time
1	0.07	0.07	1.05	0.00	0.98
2	0.12	0.12	0.23	0.00	0.11
3	0.23	0.23	0.46	0.00	0.23
4	0.23	0.23	0.26	0.00	0.03
5	0.24	0.26	0.34	0.02	0.11
6	0.26	0.34	0.40	0.08	0.14

Patient_ID	Arrival_Time	Service_Start_Time	Service_End_Time	Waiting_Time	System_Time
7	0.29	0.40	0.58	0.12	0.29
8	0.30	0.46	0.99	0.16	0.69
9	0.53	0.58	0.66	0.05	0.13
10	0.53	0.66	1.11	0.13	0.58

Average Waiting Time (3 Triage Doctors)

```r
# Compute mean waiting time for Scenario 1

mean_wait_3_docs <- mean(simulation_data$Waiting_Time) * 60

# Estimate average queue length using Little's Law (Lq = λ × Wq)

avg_queue_1 <- arrival_rate * mean(simulation_data$Waiting_Time)

# Convert to minutes

mean_waiting_time <- mean(simulation_data$Waiting_Time) * 60
mean_system_time <- mean(simulation_data$System_Time) * 60

# Average Waiting Time (3 Triage Doctors)

cat(sprintf("Mean Waiting Time (minutes): %.2f\n", mean_waiting_time))

## Mean Waiting Time (minutes): 8.82

cat(sprintf("Mean System Time (minutes): %.2f\n", mean_system_time))

## Mean System Time (minutes): 21.45
```

Average Queue Length (3 Triage Doctors)

```r
# Estimate Average Queue Length using Little's Law
# Lq = λ * Wq

# Average waiting time in HOURS
avg_wait_hours <- mean(simulation_data$Waiting_Time)

# Little's Law: average number of people in queue
avg_queue_length <- arrival_rate * avg_wait_hours

# Queue Analysis - Average Queue Length
```

```r
cat(sprintf("Arrival Rate (λ): %.2f patients/hour\n", arrival_rate))

## Arrival Rate (λ): 12.00 patients/hour

cat(sprintf("Average Waiting Time (Wq): %.4f hours (%.2f minutes)\n",
            avg_wait_hours, avg_wait_hours * 60))

## Average Waiting Time (Wq): 0.1470 hours (8.82 minutes)

cat(sprintf("Average Queue Length (Lq): %.2f patients\n", avg_queue_length))

## Average Queue Length (Lq): 1.76 patients
```

Interpretation of Results

The three-doctor system shows an average waiting time of 8.82 minutes, which is below the 10-minute threshold, indicating acceptable performance on average. However, the average system time of 21.45 minutes suggests that patients still spend a considerable total time in the emergency system.

Using Little's Law, the estimated average queue length is 1.76 patients, indicating that nearly two patients are typically waiting at any time. This reflects moderate congestion and shows that the system operates close to its capacity.

Overall, while the system meets the waiting time requirement on average, the queue length and system time suggest limited capacity and potential vulnerability to delays under higher demand.

Scenario 2: Alternative System (4 Triage Doctors)

```
# Repeat simulation with 4 doctors

num_doctors_2 <- 4

# Reinitialize
service_start2 <- numeric(n_patients)
service_end2 <- numeric(n_patients)
waiting_time2 <- numeric(n_patients)
system_time2 <- numeric(n_patients)

# Track availability of the 4 doctors
doctor_available_time2 <- rep(0, num_doctors_2)

# Simulation Loop (FCFS Logic)

for (i in 1:n_patients) {

  next_doctor <- which.min(doctor_available_time2)

  service_start2[i] <- max(arrival_time[i], doctor_available_time2[next_docto
r])

  service_end2[i] <- service_start2[i] + service_time[i]

  doctor_available_time2[next_doctor] <- service_end2[i]

  waiting_time2[i] <- service_start2[i] - arrival_time[i]

  system_time2[i] <- service_end2[i] - arrival_time[i]
}


# Store Results

simulation_data_2 <- data.frame(
  Waiting_Time = waiting_time2,
  System_Time = system_time2
)

# Compute mean waiting time for Scenario 2
mean_wait_4_docs <- mean(simulation_data_2$Waiting_Time) * 60

# Estimate average queue length using Little's Law (Lq = λ × Wq)
avg_queue_2 <- arrival_rate * mean(simulation_data_2$Waiting_Time)
```

```r
# Convert to minutes for comparison

mean_wait_3_docs <- mean(simulation_data$Waiting_Time) * 60
mean_wait_4_docs <- mean(simulation_data_2$Waiting_Time) * 60

# Average Waiting Time (4 Triage Doctors)

cat(sprintf("4 Doctors - Mean Waiting Time (minutes): %.2f\n", mean_wait_4_do
cs))

## 4 Doctors - Mean Waiting Time (minutes): 1.85

cat(sprintf("4 Doctors - Mean System Time (minutes): %.2f\n", mean(simulation
_data_2$System_Time) * 60))

## 4 Doctors - Mean System Time (minutes): 14.49

# Comparison table

comparison_simple <- data.frame(
  Metric = c(
    "Mean Waiting Time (min)",
    "Average Queue Length (patients)",
    "10 Minute Threshold Met?"
  ),

  Scenario_1_3_Doctors = c(
    round(mean_wait_3_docs, 2),
    round(avg_queue_1, 2),
    ifelse(mean_wait_3_docs <= 10, "YES", "NO")
  ),

  Scenario_2_4_Doctors = c(
    round(mean_wait_4_docs, 2),
    round(avg_queue_2, 2),
    ifelse(mean_wait_4_docs <= 10, "YES", "NO")
  )
)

knitr::kable(
  comparison_simple,
  col.names = c("Metric", "3 Doctors", "4 Doctors"),
  caption = "Key Performance Comparison: 3 vs 4 Doctors"
)
```

Key Performance Comparison: 3 vs 4 Doctors

Metric	3 Doctors	4 Doctors
Mean Waiting Time (min)	8.82	1.85
Average Queue Length (patients)	1.76	0.37
10 Minute Threshold Met?	YES	YES

Comparison and Justification

The results clearly show a significant improvement when the number of doctors is increased from three to four. The mean waiting time drops sharply from 8.82 minutes to 1.85 minutes, indicating a substantial reduction in patient delay before receiving treatment. Similarly, the average queue length decreases from 1.37 to 0.25 patients, showing that congestion in the system is almost eliminated with the additional doctor.

In terms of the hospital's mandated requirement, both scenarios satisfy the condition that the average waiting time must be below 10 minutes. However, the three-doctor system is operating much closer to the threshold, meaning it is more sensitive to fluctuations in demand. In contrast, the four-doctor system provides a much larger safety margin, with waiting times well below the limit.

Therefore, while both configurations technically meet the requirement, the addition of the fourth doctor significantly improves system performance and stability. It reduces both waiting time and queue length, making the system more robust and better suited for handling peak weekend demand.